МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Информационных технологий»

Лабораторная работа № 5
по дисциплине «Введение в информационные технологии»
по теме: «**ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ В PYTHON**»

Выполнил студент группы:
БВТ2601
Гутаев Эльдар Назирович
Проверил:

Москва 2026

Оглавление

Введение……3

Выполнение 4

Вывод..... 25

Введение

Цель и задачи работы:

Целью данной лабораторной работы является: изучить основные принципы объектно-ориентированного программирования в Python, научиться создавать классы и объекты, определять атрибуты и методы, использовать конструктор класса, применять инкапсуляцию, наследование, абстракцию и полиморфизм, а также переопределять методы родительских классов.

Задачи: В ходе выполнения лабораторной работы необходимо разработать небольшой объектно-ориентированный проект управления транспортными средствами vehicle_project: создать базовый класс Vehicle, защитить его состояние свойством property, построить иерархию наследников Car, Truck, Motorcycle и Bus, сделать базовый класс абстрактным, организовать полиморфную обработку объектов и их хранение в классе Garage через композицию, распределив классы по модулям проекта.

Выполнение

Задание 1. На рисунке 1: создан каталог проекта vehicle_project (в общей папке лабораторных работ он назван lab_05_vehicle_project) и открыт в Visual Studio Code. В корне проекта находятся main.py и garage.py, создан пакет models с файлами __init__.py и vehicle.py. На этом этапе файлы пустые: они задают структуру, которую заполняют следующие задания.

```
vehicle_project/
    main.py
    garage.py
    models/
        __init__.py
        vehicle.py
```

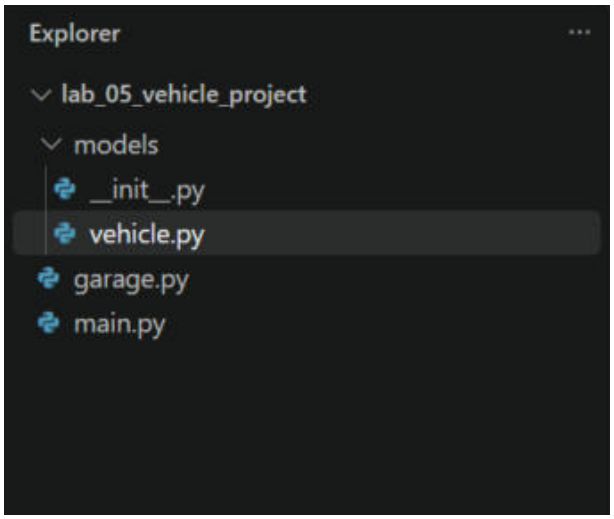


Рисунок 1. Структура проекта vehicle_project

Задание 2. На рисунке 2: в models/vehicle.py создан класс Vehicle. Конструктор __init__ принимает brand, model и year и сохраняет их в атрибуты экземпляра через self, атрибут _speed получает значение 0. В main.py созданы два объекта: Toyota Camry 2024 и Lada Vesta 2023. Проверка car1 is car2 вернула False, то есть это два разных экземпляра одного класса. После присваивания car1._speed = 60 скорость car1 стала 60, а у car2 осталась

0: каждый объект хранит собственное состояние, потому что self в конструкторе указывает именно на создаваемый объект.

```
class Vehicle:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year
        self._speed = 0

from models.vehicle import Vehicle

car1 = Vehicle("Toyota", "Camry", 2024)
car2 = Vehicle("Lada", "Vesta", 2023)

print("Первый объект:", car1.brand, car1.model, car1.year,
car1._speed)
print("Второй объект:", car2.brand, car2.model, car2.year,
car2._speed)
print("car1 is car2:", car1 is car2)

car1._speed = 60
print("После изменения скорости только у car1:")
print("Скорость car1:", car1._speed)
print("Скорость car2:", car2._speed)
```

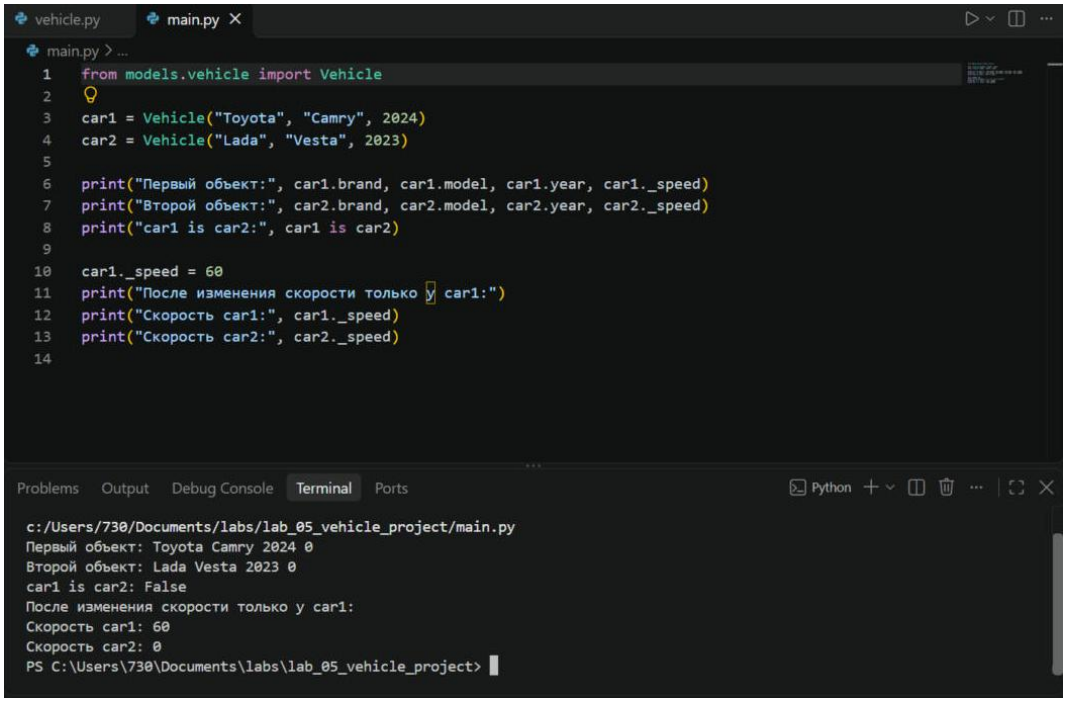


Рисунок 2. Класс Vehicle и два независимых объекта

Задание 3. В класс Vehicle добавлены методы accelerate(value), brake(value), stop() и get_info(). accelerate() увеличивает скорость на переданное значение; brake() уменьшает её, но благодаря max(0, ...) результат не становится меньше нуля; stop() обнуляет скорость; get_info() возвращает марку, модель и

год. На рисунке 3 показан main.py, на рисунке 3.1 — полный вывод в развёрнутом терминале: скорость меняется 0 → 50 → 80 → 60, затем brake(100) даёт 0, а не −40; после accelerate(40) и stop() скорость снова 0.

```
class Vehicle:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year
        self._speed = 0

    def accelerate(self, value):
        self._speed += value

    def brake(self, value):
        self._speed = max(0, self._speed - value)

    def stop(self):
        self._speed = 0

    def get_info(self):
        return f"{self.brand} {self.model}, {self.year} г."
```

```
from models.vehicle import Vehicle

car = Vehicle("Toyota", "Camry", 2024)
print(car.get_info())
print("Начальная скорость:", car._speed)

car.accelerate(50)
print("После accelerate(50):", car._speed)
car.accelerate(30)
print("После accelerate(30):", car._speed)
car.brake(20)
print("После brake(20):", car._speed)
car.brake(100)
print("После brake(100):", car._speed)

car.accelerate(40)
print("После accelerate(40):", car._speed)
car.stop()
print("После stop():", car._speed)
```

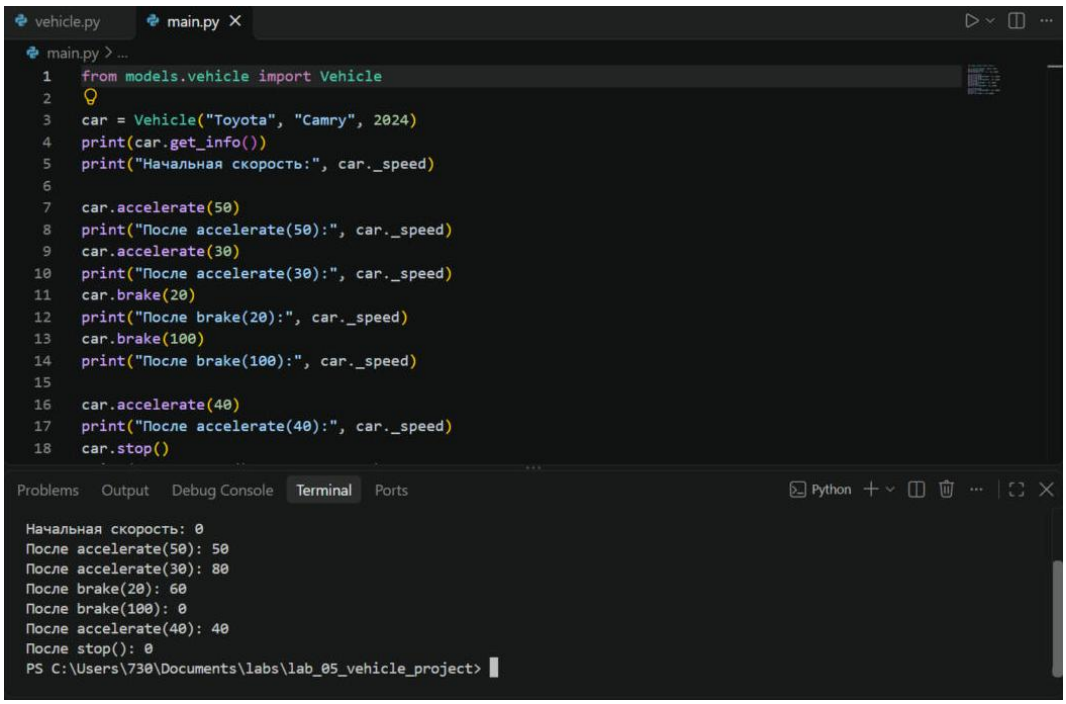


Рисунок 3. Демонстрация методов в main.py

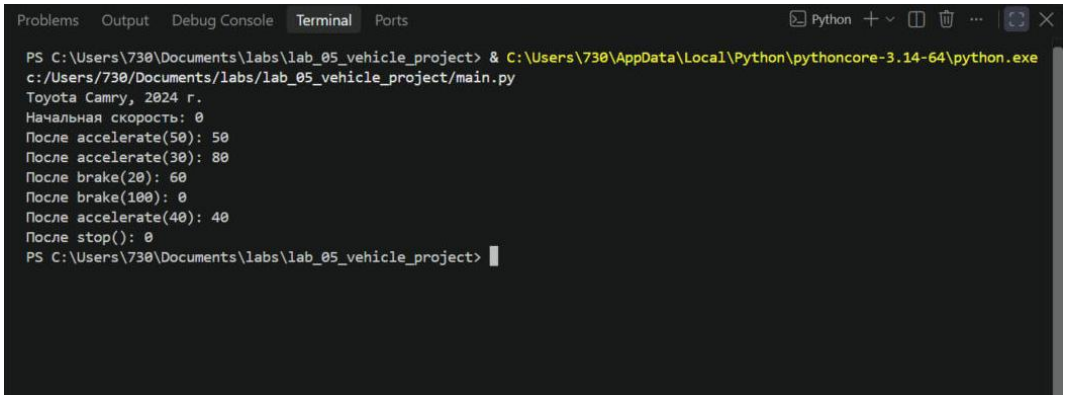


Рисунок 3.1. Результат работы методов accelerate(), brake() и stop()

Задание 4. На рисунке 4: добавлено свойство speed. Геттер с декоратором @property возвращает _speed, сеттер @speed.setter проверяет значение и при отрицательном выбрасывает ValueError. Методы accelerate(), brake() и stop() теперь меняют скорость через self.speed, поэтому любая запись проходит проверку. Присваивание car.speed = 80 выглядит как работа с обычным атрибутом; далее accelerate(20) даёт 100, brake(30) — 70. Попытка car.speed = -10 перехвачена блоком try/except, выведено «Ошибка: Скорость не может

быть отрицательной», а скорость осталась 70: сеттер отклонил значение до записи в _speed.

```
class Vehicle:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year
        self._speed = 0

    @property
    def speed(self):
        return self._speed

    @speed.setter
    def speed(self, value):
        if value < 0:
            raise ValueError("Скорость не может быть
отрицательной")
        self._speed = value

    def accelerate(self, value):
        self.speed = self.speed + value

    def brake(self, value):
        self.speed = max(0, self.speed - value)

    def stop(self):
        self.speed = 0

    def get_info(self):
        return f"{self.brand} {self.model}, {self.year} г."

from models.vehicle import Vehicle

car = Vehicle("Toyota", "Camry", 2024)
print(car.get_info())

car.speed = 80
print("После car.speed = 80:", car.speed)
car.accelerate(20)
print("После accelerate(20):", car.speed)
car.brake(30)
print("После brake(30):", car.speed)

try:
    car.speed = -10
except ValueError as error:
    print("Ошибка:", error)

print("Скорость не изменилась:", car.speed)
```

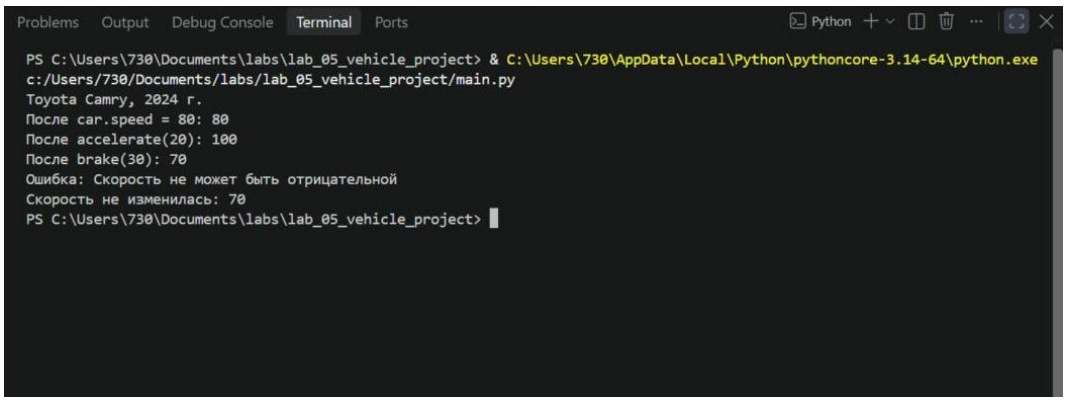


Рисунок 4. Свойство speed и обработка отрицательной скорости

Задание 5. На рисунке 5: в Vehicle реализован метод __str__(), который возвращает марку, модель, год и текущую скорость. Команда print(car1) выводит «Toyota Camry (2024), скорость: 60 км/ч» вместо стандартного <models.vehicle.Vehicle object at ...>. Тот же текст получается при явном

вызове str(car1) и при подстановке объекта в f-строку: print(), str() и f-строки вызывают __str__() автоматически.

```
    def __str__(self):
        return f"{self.brand} {self.model} ({self.year}),
скорость: {self.speed} км/ч"

from models.vehicle import Vehicle

car1 = Vehicle("Toyota", "Camry", 2024)
car2 = Vehicle("Lada", "Vesta", 2023)

car1.accelerate(60)

print(car1)
print(car2)
print("Через str():", str(car1))
print(f"В f-строке: {car2}")
```

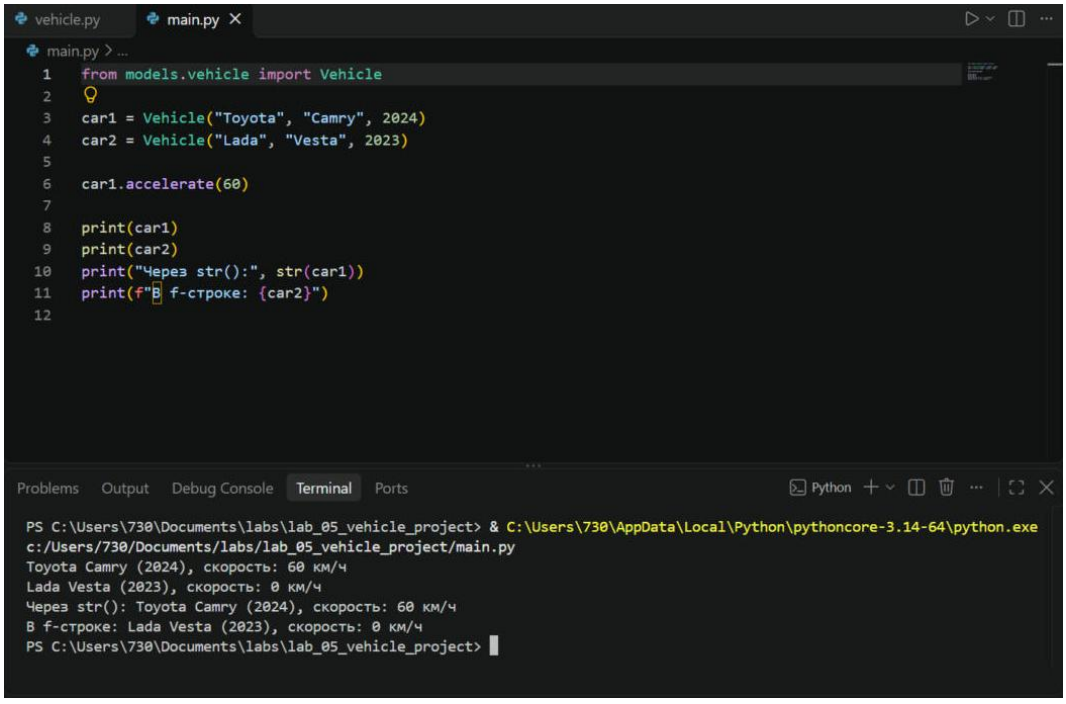


Рисунок 5. Текстовое представление объекта через __str__()

Задание 6. На рисунке 6: создан файл models/car.py с классом Car(Vehicle). Конструктор принимает дополнительный параметр doors, общую часть объекта (brand, model, year, _speed) инициализирует вызовом super().__init__(), затем сохраняет собственный атрибут doors. Метод get_info() переопределён: он берёт строку родительской реализации через super().get_info() и дописывает тип и число дверей. Vehicle.get_info() выводит «Lada Vesta, 2023 г.», а Car.get_info() — «Автомобиль: Toyota Camry, 2024 г.,

дверей: 4». Методы accelerate(), brake() и __str__() в Car не объявлялись, но работают, так как унаследованы от Vehicle (90 − 30 = 60 км/ч).

```
from models.vehicle import Vehicle


class Car(Vehicle):
    def __init__(self, brand, model, year, doors):
        super().__init__(brand, model, year)
        self.doors = doors

    def get_info(self):
        return f"Автомобиль: {super().get_info()}, дверей:
{self.doors}"

from models.car import Car
from models.vehicle import Vehicle

vehicle = Vehicle("Lada", "Vesta", 2023)
car = Car("Toyota", "Camry", 2024, 4)

print("Vehicle.get_info():", vehicle.get_info())
print("Car.get_info():", car.get_info())

car.accelerate(90)
car.brake(30)
print("Унаследованные accelerate() и brake(), скорость:",
car.speed)
print("Унаследованный __str__():", car)
print("Собственный атрибут doors:", car.doors)
```

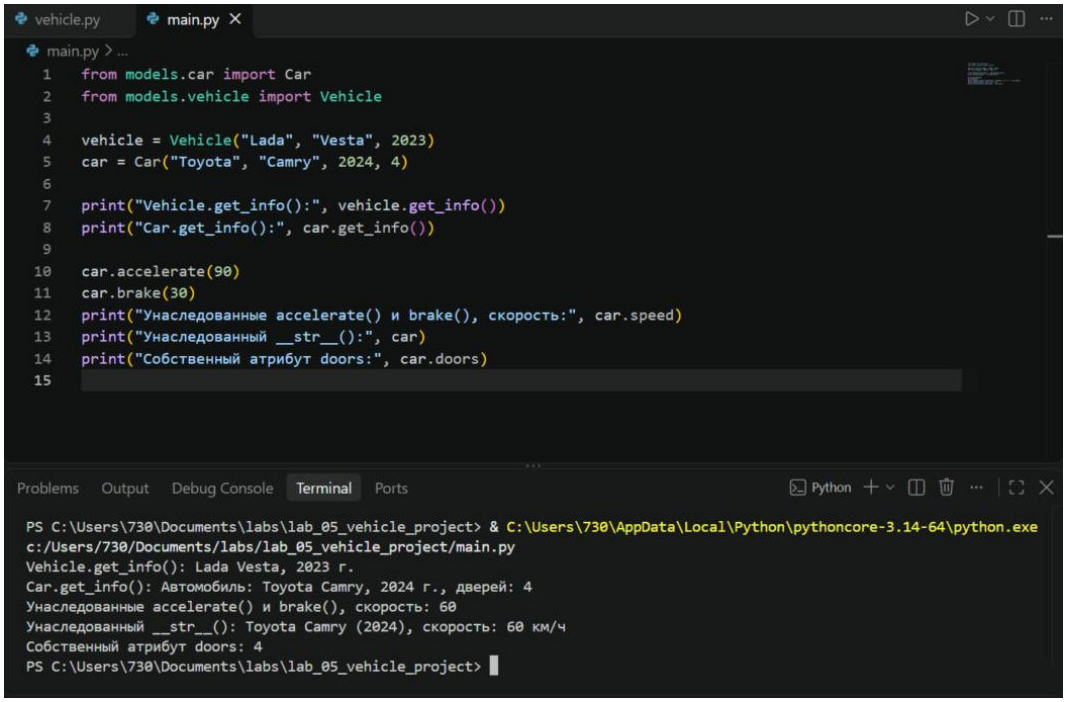


Рисунок 6. Класс Car: наследование и переопределение get_info()

Задание 7. На рисунке 7: созданы models/truck.py и models/motorcycle.py. Truck хранит грузоподъёмность max_load, Motorcycle — признак has_sidecar. Оба класса наследуются от Vehicle, вызывают super().__init__() и переопределяют get_info() с учётом своих атрибутов: для грузовика выводится «грузоподъёмность: 20 т», для мотоциклов — «без коляски» (Yamaha) и «с коляской» (Урал). Унаследованные accelerate() и __str__()

работают и в новых классах: КАМАЗ разогнан до 70 км/ч, Yamaha — до 120 км/ч.

```
from models.vehicle import Vehicle


class Truck(Vehicle):
    def __init__(self, brand, model, year, max_load):
        super().__init__(brand, model, year)
        self.max_load = max_load

    def get_info(self):
        return f"Грузовик: {super().get_info()},
грузоподъёмность: {self.max_load} т"
```

```
from models.vehicle import Vehicle


class Motorcycle(Vehicle):
    def __init__(self, brand, model, year, has_sidecar):
        super().__init__(brand, model, year)
        self.has_sidecar = has_sidecar

    def get_info(self):
        sidecar = "с коляской" if self.has_sidecar else "без
коляски"
        return f"Мотоцикл: {super().get_info()}, {sidecar}"
```

```
from models.car import Car
from models.motorcycle import Motorcycle
from models.truck import Truck

car = Car("Toyota", "Camry", 2024, 4)
truck = Truck("КАМАЗ", "54901", 2022, 20)
moto1 = Motorcycle("Yamaha", "MT-07", 2021, False)
moto2 = Motorcycle("Урал", "Gear-Up", 2020, True)

print(car.get_info())
print(truck.get_info())
print(moto1.get_info())
print(moto2.get_info())

truck.accelerate(70)
moto1.accelerate(120)
print(truck)
print(moto1)
```

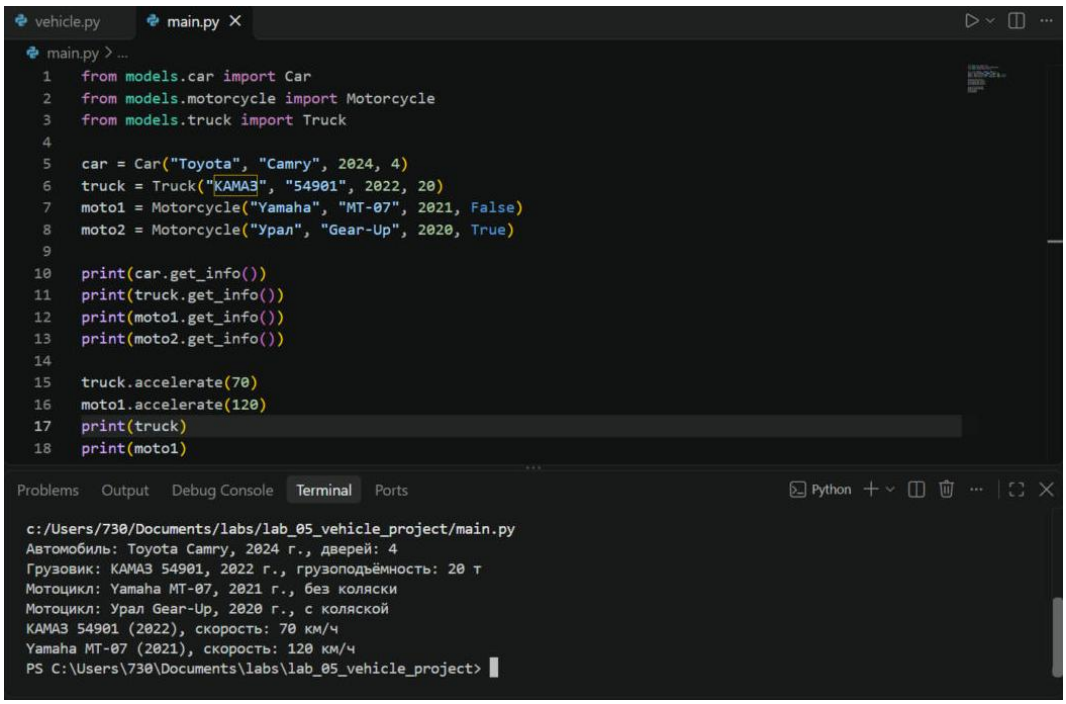


Рисунок 7. Классы Truck и Motorcycle

Задание 8. На рисунке 8: класс Vehicle теперь наследуется от ABC и содержит абстрактный метод move() с декоратором @abstractmethod; в Car, Truck и Motorcycle метод move() реализован по-своему. Попытка создать Vehicle("Lada", "Vesta", 2023) завершилась исключением TypeError: «Can't instantiate abstract class Vehicle without an implementation for abstract method 'move'». Экземпляр Vehicle создать нельзя, потому что у класса остался нереализованный абстрактный метод: ABC запрещает создавать объекты таких классов, и Vehicle теперь задаёт только общий контракт. Объекты

наследников, реализовавших move(), создаются без ошибок и выводят свои сообщения о движении.

```
from abc import ABC, abstractmethod


class Vehicle(ABC):
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year
        self._speed = 0

    @property
    def speed(self):
        return self._speed

    @speed.setter
    def speed(self, value):
        if value < 0:
            raise ValueError("Скорость не может быть
отрицательной")
        self._speed = value

    def accelerate(self, value):
        self.speed = self.speed + value

    def brake(self, value):
        self.speed = max(0, self.speed - value)

    def stop(self):
        self.speed = 0

    def get_info(self):
        return f"{self.brand} {self.model}, {self.year} г."

    @abstractmethod
    def move(self):
        pass

    def __str__(self):
        return f"{self.brand} {self.model} ({self.year}),
скорость: {self.speed} км/ч"

from models.vehicle import Vehicle


class Car(Vehicle):
    def __init__(self, brand, model, year, doors):
        super().__init__(brand, model, year)
        self.doors = doors

    def get_info(self):
        return f"Автомобиль: {super().get_info()}, дверей:
{self.doors}"
```

```
    def move(self):
        return f"Автомобиль {self.brand} едет по дороге"

# models/truck.py
    def move(self):
        return f"Грузовик {self.brand} везёт груз по трассе"

# models/motorcycle.py
    def move(self):
        return f"Мотоцикл {self.brand} мчится по дороге"

from models.car import Car
from models.motorcycle import Motorcycle
from models.truck import Truck
from models.vehicle import Vehicle

try:
    vehicle = Vehicle("Lada", "Vesta", 2023)
except TypeError as error:
    print("Ошибка при создании Vehicle:")
    print(error)

car = Car("Toyota", "Camry", 2024, 4)
truck = Truck("КАМАЗ", "54901", 2022, 20)
moto = Motorcycle("Yamaha", "MT-07", 2021, False)

print(car.move())
print(truck.move())
print(moto.move())
```

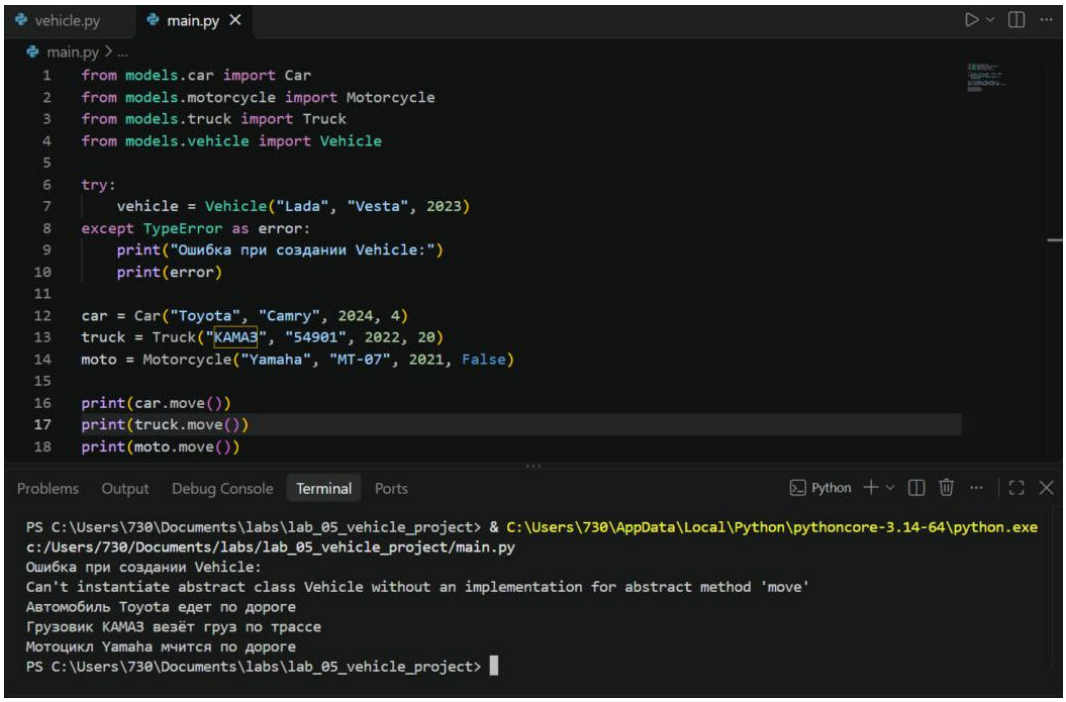


Рисунок 8. Абстрактный класс Vehicle и метод move()

Задание 9. На рисунке 9: список vehicles содержит объекты Car, Truck и Motorcycle. В одном цикле для каждого объекта вызываются get_info() и

move() без isinstance() и сравнения типов, а вывод у всех разный. Python ищет метод сначала в классе самого объекта, затем по цепочке наследования (MRO). В Car, Truck и Motorcycle есть свои get_info() и move(), поэтому вызывается реализация фактического класса объекта. Это и есть полиморфизм: один интерфейс, разное поведение.

```
from models.car import Car
from models.motorcycle import Motorcycle
from models.truck import Truck

vehicles = [
    Car("Toyota", "Camry", 2024, 4),
    Truck("КАМАЗ", "54901", 2022, 20),
    Motorcycle("Урал", "Gear-Up", 2020, True),
]

for vehicle in vehicles:
    print(vehicle.get_info())
    print("   ", vehicle.move())
```

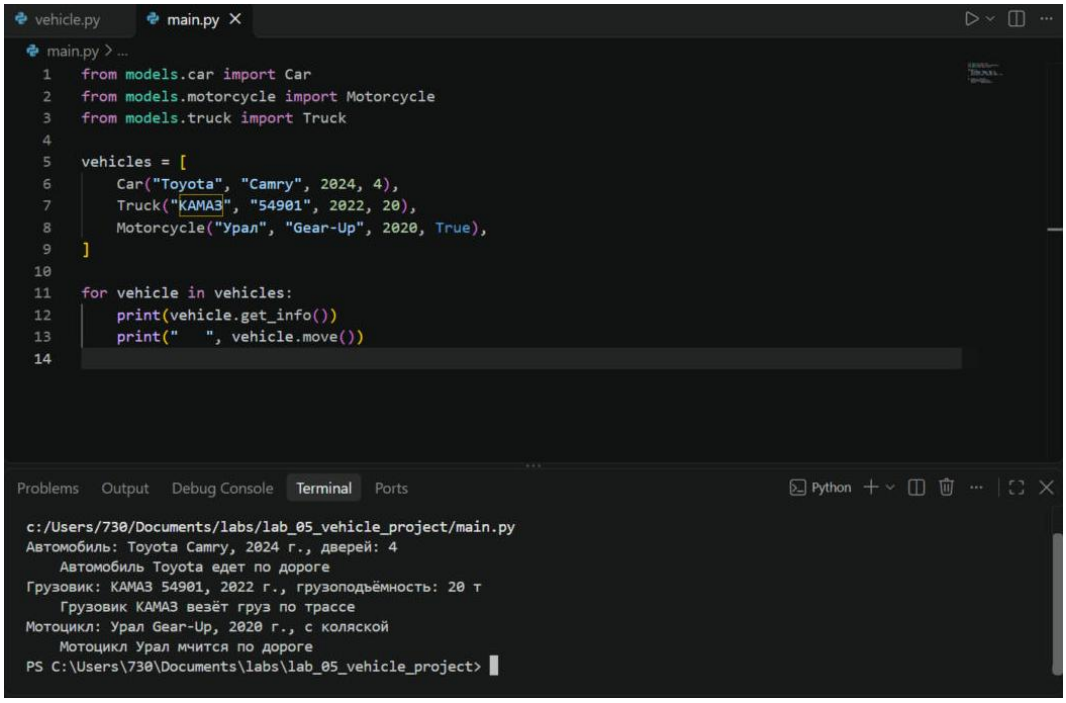


Рисунок 9. Полиморфный вызов get_info() и move()

Задание 10. В garage.py создан класс Garage. Конструктор создаёт пустой список vehicles; add_vehicle() и remove_vehicle() добавляют и удаляют объект, show_all() выводит пронумерованный список через get_info(), find_by_brand() возвращает транспорт указанной марки без учёта регистра. На рисунке 10 показан main.py, на рисунке 10.1 — полный вывод: в гараж

добавлены две Toyota, грузовик и мотоцикл, поиск по «toyota» нашёл Camry и Corolla, после удаления грузовика осталось 3 транспортных средства. Garage не наследуется от Vehicle: он хранит объекты транспорта

(композиция) и не содержит отдельной логики для Car, Truck и Motorcycle, а пользуется только их общим интерфейсом.

```
class Garage:
    def __init__(self):
        self.vehicles = []

    def add_vehicle(self, vehicle):
        self.vehicles.append(vehicle)

    def remove_vehicle(self, vehicle):
        if vehicle in self.vehicles:
            self.vehicles.remove(vehicle)
        else:
            print("Этого транспортного средства нет в гараже")

    def show_all(self):
        print("Транспорт в гараже:", len(self.vehicles))
        for number, vehicle in enumerate(self.vehicles,
start=1):
            print(f"{number}. {vehicle.get_info()}")

    def find_by_brand(self, brand):
        return [
            vehicle for vehicle in self.vehicles
            if vehicle.brand.lower() == brand.lower()
        ]

from garage import Garage
from models.car import Car
from models.motorcycle import Motorcycle
from models.truck import Truck

camry = Car("Toyota", "Camry", 2024, 4)
corolla = Car("Toyota", "Corolla", 2019, 5)
truck = Truck("КАМАЗ", "54901", 2022, 20)
moto = Motorcycle("Yamaha", "MT-07", 2021, False)

garage = Garage()
for vehicle in [camry, corolla, truck, moto]:
    garage.add_vehicle(vehicle)
garage.show_all()

print("Поиск по марке toyota:")
for vehicle in garage.find_by_brand("toyota"):
    print("-", vehicle)

garage.remove_vehicle(truck)
print("После удаления грузовика:")
garage.show_all()
```

```python
from garage import Garage
from models.car import Car
from models.motorcycle import Motorcycle
from models.truck import Truck

camry = Car("Toyota", "Camry", 2024, 4)
corolla = Car("Toyota", "Corolla", 2019, 5)
truck = Truck("КАМАЗ", "54901", 2022, 20)
moto = Motorcycle("Yamaha", "MT-07", 2021, False)

garage = Garage()
for vehicle in [camry, corolla, truck, moto]:
    garage.add_vehicle(vehicle)
garage.show_all()

print("Поиск по марке toyota:")
for vehicle in garage.find_by_brand("toyota"):
    print("-", vehicle)
```

```
- Toyota Camry (2024), скорость: 0 км/ч
- Toyota Corolla (2019), скорость: 0 км/ч
После удаления грузовика:
Транспорт в гараже: 3
1. Автомобиль: Toyota Camry, 2024 г., дверей: 4
2. Автомобиль: Toyota Corolla, 2019 г., дверей: 5
3. Мотоцикл: Yamaha MT-07, 2021 г., без коляски
PS C:\Users\730\Documents\labs\lab_05_vehicle_project>
```

Рисунок 10. Использование класса Garage в main.py

```
PS C:\Users\730\Documents\labs\lab_05_vehicle_project> & C:\Users\730\AppData\Local\Python\pythoncore-3.14-64\python.exe
c:/Users/730/Documents/labs/lab_05_vehicle_project/main.py
Транспорт в гараже: 4
1. Автомобиль: Toyota Camry, 2024 г., дверей: 4
2. Автомобиль: Toyota Corolla, 2019 г., дверей: 5
3. Грузовик: КАМАЗ 54901, 2022 г., грузоподъёмность: 20 т
4. Мотоцикл: Yamaha MT-07, 2021 г., без коляски
Поиск по марке toyota:
- Toyota Camry (2024), скорость: 0 км/ч
- Toyota Corolla (2019), скорость: 0 км/ч
После удаления грузовика:
Транспорт в гараже: 3
1. Автомобиль: Toyota Camry, 2024 г., дверей: 4
2. Автомобиль: Toyota Corolla, 2019 г., дверей: 5
3. Мотоцикл: Yamaha MT-07, 2021 г., без коляски
PS C:\Users\730\Documents\labs\lab_05_vehicle_project>
```

Рисунок 10.1. Добавление, поиск и удаление в гараже

Задание 11. На рисунке 11: итоговая структура проекта — main.py, garage.py и пакет models с модулями __init__.py, vehicle.py, car.py, truck.py и motorcycle.py. В models/__init__.py классы импортированы на уровень пакета, поэтому в main.py достаточно строки from models import Car, Motorcycle, Truck. Демонстрационный сценарий перенесён в функцию main(), которая вызывается под условием if __name__ == "__main__". На рисунке 11.1: проект запущен из корневого каталога без ошибок: три объекта разогнаны и

выполнили move(), добавлены в гараж и выведены show_all(), поиск по марке Toyota нашёл Camry со скоростью 60 км/ч.

```
vehicle_project/
    main.py
    garage.py
    models/
        __init__.py
        vehicle.py
        car.py
        truck.py
        motorcycle.py
```

```
from models.car import Car
from models.motorcycle import Motorcycle
from models.truck import Truck
from models.vehicle import Vehicle

__all__ = ["Vehicle", "Car", "Truck", "Motorcycle"]
```

```
from garage import Garage
from models import Car, Motorcycle, Truck


def main():
    vehicles = [
        Car("Toyota", "Camry", 2024, 4),
        Truck("КАМАЗ", "54901", 2022, 20),
        Motorcycle("Yamaha", "MT-07", 2021, False),
    ]

    print("Движение транспорта:")
    for vehicle in vehicles:
        vehicle.accelerate(60)
        print("-", vehicle.move())

    garage = Garage()
    for vehicle in vehicles:
        garage.add_vehicle(vehicle)
    garage.show_all()

    print("Поиск по марке Toyota:")
    for vehicle in garage.find_by_brand("Toyota"):
        print("-", vehicle)


if __name__ == "__main__":
    main()
```

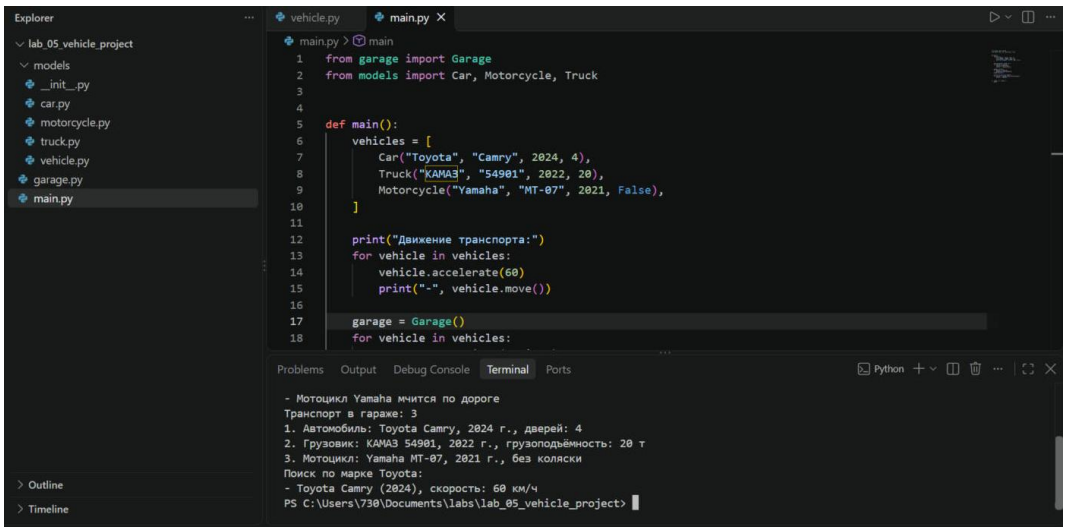


Рисунок 11. Итоговая структура проекта и функция main()

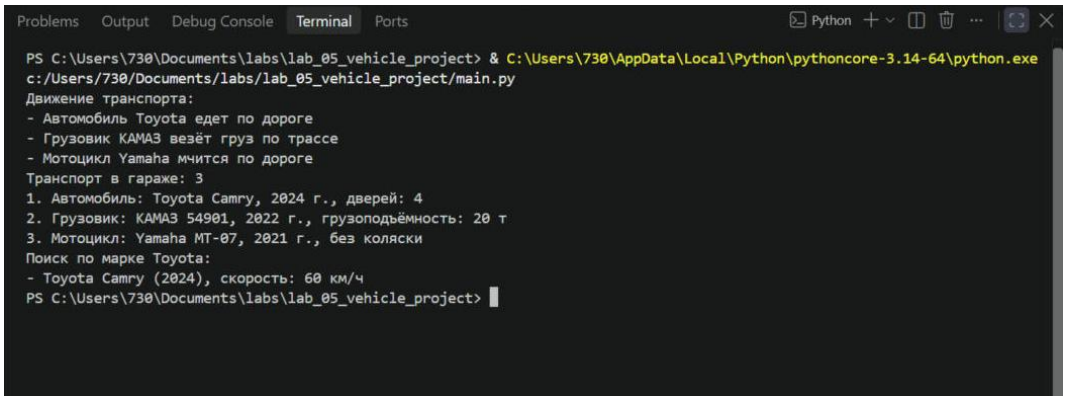


Рисунок 11.1. Запуск итогового проекта

Задание 12. Для самостоятельной работы выбран класс Bus (автобус). Он наследуется от Vehicle, имеет два собственных атрибута — seats (число мест) и route_number (номер маршрута), — инициализирует общую часть через super().__init__(), переопределяет get_info() и реализует абстрактный метод move() (рисунок 12). Класс добавлен в models/__init__.py, а в main.py автобус ЛиАЗ 5292 включён в общий список vehicles (рисунок 12.1). На рисунке 12.2 автобус разогнан тем же циклом, вывел «Автобус ЛиАЗ едет по маршруту № 144», попал в гараж и показан методом show_all() вместе с остальными; поиск по марке ЛиАЗ его нашёл. Файл garage.py и цикл в main() не менялись:

новый тип обрабатывается через общий интерфейс Vehicle без проверок типа.

```
from models.vehicle import Vehicle


class Bus(Vehicle):
    def __init__(self, brand, model, year, seats, route_number):
        super().__init__(brand, model, year)
        self.seats = seats
        self.route_number = route_number

    def get_info(self):
        return (
            f"Автобус: {super().get_info()}, мест: {self.seats},
"
            f"маршрут № {self.route_number}"
        )

    def move(self):
        return f"Автобус {self.brand} едет по маршруту №
{self.route_number}"

from models.bus import Bus
from models.car import Car
from models.motorcycle import Motorcycle
from models.truck import Truck
from models.vehicle import Vehicle

__all__ = ["Vehicle", "Car", "Truck", "Motorcycle", "Bus"]

from garage import Garage
from models import Bus, Car, Motorcycle, Truck


def main():
    vehicles = [
        Car("Toyota", "Camry", 2024, 4),
        Truck("КАМАЗ", "54901", 2022, 20),
        Motorcycle("Yamaha", "MT-07", 2021, False),
        Bus("ЛиАЗ", "5292", 2023, 110, 144),
    ]

    print("Движение транспорта:")
    for vehicle in vehicles:
        vehicle.accelerate(60)
        print("-", vehicle.move())

    garage = Garage()
    for vehicle in vehicles:
        garage.add_vehicle(vehicle)
    garage.show_all()

    print("Поиск по марке ЛиАЗ:")
    for vehicle in garage.find_by_brand("ЛиАЗ"):
        print("-", vehicle)
```

```
if __name__ == "__main__":
    main()
```

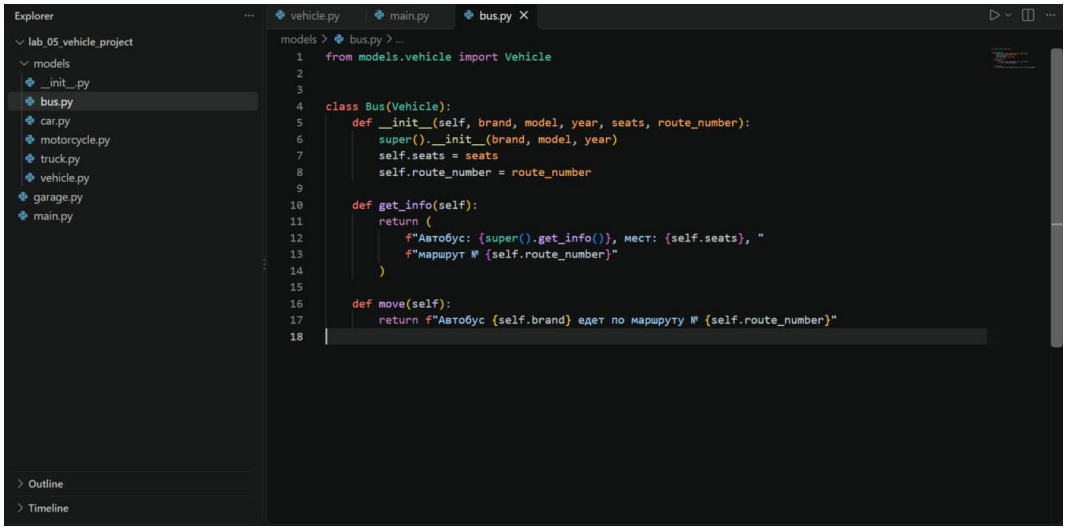


Рисунок 12. Класс Bus

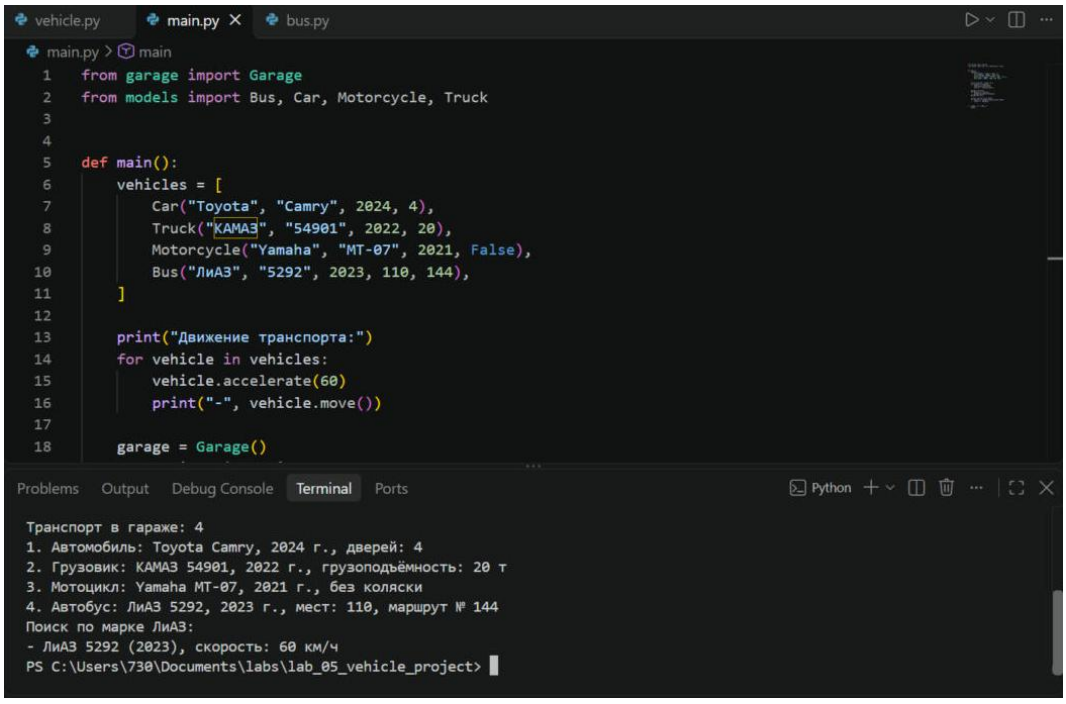


Рисунок 12.1. Добавление Bus в общий список vehicles

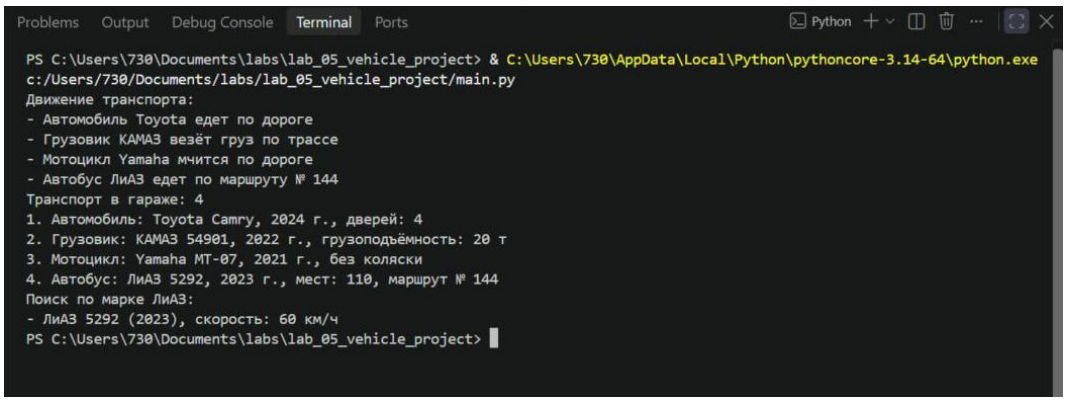


Рисунок 12.2. Работа Bus в общем списке и в гараже

Вывод

В ходе лабораторной работы изучены основы объектно-ориентированного программирования в Python. Создан класс Vehicle с конструктором __init__, атрибутами экземпляра и методами accelerate(), brake(), stop() и get_info(). Опыт с двумя объектами показал разницу между классом и объектом: экземпляры одного класса хранят независимое состояние, а self указывает на объект, для которого вызван метод.

Применена инкапсуляция: скорость хранится в атрибуте _speed, а доступ к ней идёт через свойство speed, сеттер которого не пропускает отрицательные значения и выбрасывает ValueError. Метод __str__() дал удобное текстовое представление объекта, которое автоматически используется в print(), str() и f-строках.

Построена иерархия наследников Car, Truck и Motorcycle: они вызывают super().__init__(), добавляют собственные атрибуты и переопределяют get_info(). Класс Vehicle сделан абстрактным с помощью ABC и @abstractmethod, и попытка создать его экземпляр закончилась TypeError. Полиморфный цикл по списку vehicles вызвал нужные реализации move() и get_info() без проверок типа.

Класс Garage реализует композицию: он хранит объекты транспорта и работает с ними только через общий интерфейс. Классы разнесены по модулям пакета models, сценарий оформлен в функции main() с точкой входа if

__name__ == "__main__". Самостоятельно добавленный класс Bus встроился в общий список и в гараж без изменения Garage, что подтверждает удобство общего интерфейса базового класса.